

模块接口协作约定

项目：Qt Knight
接口环境：C++17、Qt 6.12、MSVC2022、64 位
依据：[项目总体架构](#)、[业务逻辑与数据流](#)

1 使用范围与调用边界

本文只定义目标接口、类型和调用规则，不记录开发进度。两人写代码时，先共同确认 `KnightContracts.h` 和各模块公开 `.h`，再分别编写 `.cpp`。函数栏只写 C++ 函数声明；开发状态另写文档。

调用方向固定为：`KnightUI.exe` → `KnightGame.dll` → 七个底层 DLL。界面只包含 `KnightGame.h`；业务层包含底层模块的公开头文件；底层模块之间**不包含对方头文件、不链接对方导入库、不直接调用对方函数**。下层返回共享值对象，业务层接收后再作为另一次调用的输入。

动态库	中文模块名	对业务层公开的头文件	私有数据
<code>KnightGame.dll</code>	游戏协调与业务	<code>KnightGame.h</code> ，只供界面包含	<code>GameHandle</code> 、会话、输入队列
<code>KnightState.dll</code>	对象与状态	<code>KnightState.h</code>	<code>World</code> 、实体注册表
<code>KnightMap.dll</code>	地图与碰撞	<code>KnightMap.h</code>	<code>MapHandle</code> 、空间索引
<code>KnightMotion.dll</code>	运动与物理	<code>KnightMotion.h</code>	只使用本次调用的临时变量
<code>KnightCombat.dll</code>	战斗与陷阱	<code>KnightCombat.h</code>	只使用本次调用的临时变量
<code>KnightAI.dll</code>	敌人决策	<code>KnightAI.h</code>	只使用本次调用的临时变量
<code>KnightContent.dll</code>	关卡与配置	<code>KnightContent.h</code>	文件解析缓存
<code>KnightSave.dll</code>	存档与数据库	<code>KnightSave.h</code>	<code>SaveHandle</code> 、数据库连接

`KnightContracts.h` 是上述目标共用的**类型头文件**，不对应一个 DLL。公开函数用本 DLL 的导出宏声明；“模块内部函数”只在本模块 `.cpp` 或私有头文件声明，不能由业务层调用。

2 先读懂函数签名、返回值与共享类型

2.1 `Result<CombatResult>` 到底是什么

以 `Result<CombatResult> evaluateAttacks(const CombatInput& input) noexcept` 为例：函数名是 `evaluateAttacks`；输入参数 `input` 的类型是 `CombatInput`；整个函数的**返回类型**是 `Result<CombatResult>`。`Result<T>` 是通用返回包装，包含调用是否成功的 `status` 和成功时的 `value`。这里把 `T` 换成 `CombatResult`，所以 `value` 的类型就是 `CombatResult`。`CombatResult` 再包含状态变更候选、伤害候选和事件候选。

以下是返回封装与战斗结果的关键代码节选，均放在 `src/contracts/KnightContracts.h`，并由使用它们的公开头文件包含。实际头文件要先定义 `ChangeBatch`、`DamageCandidate`、`EventBatch`，再定义 `CombatResult`：

```
enum class StatusCode : std::uint8_t {
    Ok = 0, InvalidArgument, InvalidState, NotFound,
    InvalidFormat, VersionMismatch, IoFailure, OutOfMemory, InternalFailure
};

struct Status {
    StatusCode code = StatusCode::Ok;
    std::array<char, 256> messageUtf8{};
};

template<class T>
struct Result {
    Status status{};
    T value{};
};

struct CombatResult {
    ChangeBatch changes;
    std::vector<DamageCandidate> damage;
    EventBatch events;
};
```

调用方先检查 `result.status.code == StatusCode::Ok`，成功后才能使用 `result.value`。`messageUtf8` 以零字节结尾。若函数直接返回 `Status`，它只报告成功或失败，不另带数据值。若返回 `void`，本接口明确说明其释放等副作用。`const T&` 是本次调用只读输入；`T*` 是句柄或可空输入；可写 `T&` 则是输出参数或被修改的私有对象。公开函数的异常必须在所属 DLL 内处理，不能跨 DLL 抛出。

调用端读取结果的方式如下：这里的 `result.value` 是 `CombatResult` 结构体，`result.value.damage` 是 `std::vector<DamageCandidate>`：

```
knight::Result<knight::CombatResult> result = knight::evaluateAttacks(input);
if (result.status.code == knight::StatusCode::Ok) {
    const knight::CombatResult& combat = result.value;
    const std::vector<knight::DamageCandidate>& hits = combat.damage;
    // 用 hits 继续组装本帧伤害候选。
} else {
    // 读取 result.status.messageUtf8; 不要使用 result.value。
}
```

2.2 共享类型在哪里定义、每个结构体装什么

下表所有类型、别名和枚举统一声明在 `src/contracts/knightContracts.h`。该头文件需要包含 `<array>`、`<cstdint>`、`<string>`、`<vector>`、`<unordered_map>`、`<variant>`。表中的“成员”是目标 C++ 字段类型和名称，不是模糊的业务说明。`std::vector<T>` 为值语义列表，`std::string` 存 UTF-8；两个开发者必须使用相同的 MSVC、运行时库、位数和构建配置。

类型	C++ 成员或别名
EntityId、FrameId、CommandId	均为 std::uint64_t
SlotId	std::int64_t
LevelId、ResourceId、ContentRoot、DatabasePath	均为 std::string
Vec2	float x、float y
Rect	float x、float y、float width、float height
Action	enum class: MoveLeft、MoveRight、Jump、Attack、Interact
CommandPhase	enum class: Press、Release、Trigger
EntityKind、AIState、AttackPhase	EntityKind: Player、Enemy、Pickup; AIState: None、Patr
PlayerCommand	CommandId id; Action action; CommandPhase phase; float d
GameConfig	ContentRoot contentRoot; DatabasePath databasePath; Level
AttackInstance	std::uint64_t id; EntityId owner; FrameId startedFrame; ,

类型	C++ 成员或别名
EntitySnapshot	EntityId id、target; EntityKind kind; Vec2 position、vel active、grounded; FrameId invulnerableUntil、attackReadyF spriteKey、equippedAttackId
ProgressState	LevelId levelId; std::string checkpointId; int health、e clearedObjectives
worldSnapshot	LevelId levelId; FrameId frame; std::vector<EntitySnapsho levelComplete
SaveData	std::uint32_t formatVersion; SlotId slotId; ProgressState
SlotSeed	ProgressState initialProgress
SlotSummary、SlotSummaryBatch	摘要: SlotId slotId、LevelId levelId、std::string checkpo

类型	C++ 成员或别名	含义及约束
PlayerRules	float moveSpeed、jumpSpeed、gravity、maxFallSpeed; int maxJumps、 maxHealth、maxEnergy; Rect collider; ResourceId spriteKey、 defaultAttackId	玩家出生时的 数值、碰撞盒 和默认资源; 当前血量等运 行值不写回原 型
EnemyRules	float moveSpeed、detectRadius、attackRange; FrameId attackCooldown; int maxHealth、maxEnergy; Rect collider; ResourceId spriteKey、 attackId、behaviorId	对应敌人原型 的初始属性与 决策参数
AttackRules	Rect hitBox; int damage; FrameId windupFrames、activeFrames、 recoveryFrames、cooldownFrames	攻击判定参数
TrapRules	int damage; FrameId periodFrames、activeFrames	陷阱周期和伤 害
PrototypeCatalog	PlayerRules player; std::unordered_map<ResourceId, EnemyRules> enemies; std::unordered_map<ResourceId, AttackRules> attacks; std::unordered_map<ResourceId, TrapRules> traps	已解析的数值 原型, 不能含 内容模块私有 指针
TileLayer	int width、height; std::vector<int> tiles	图块按行存 储, 数量等于 width × height
EntitySpawn	EntityId id; EntityKind kind; ResourceId prototypeId、spriteKey; Vec2 position; Rect collider; std::vector<Vec2> patrolPoints	关卡初始敌人 与拾取物; 拾 取物必须提供 碰撞盒, 巡逻 点只供敌人使 用
TriggerKind	enum class: Hazard、Pickup、Checkpoint、Door、Exit	地图触发区类 别

类型	C++ 成员或别名	含义及约束
TriggerDefinition	std::string id; TriggerKind kind; Rect area; ResourceId prototypeId; EntityId linkedEntity	陷阱、物品、检查点、门、出口的区域定义；拾取触发区关联一个物品实体
LevelDefinition	std::uint32_t formatVersion; LevelId id、nextLevelId; float tileSize; TileLayer collisionLayer; std::vector<TileLayer> visualLayers; Vec2 playerSpawn; std::vector<EntitySpawn> spawns; std::vector<TriggerDefinition> triggers; std::vector<std::string> objectives; PrototypeCatalog prototypes	内容模块完成解析、校验后的关卡值对象

类型	C++ 成员或别名
Intent、IntentBatch	意图: EntityId entity、float horizontal、bool jump、bool attack; 批次:
MotionItem、MotionBatch	项: EntityId entity、Vec2 proposedPosition、proposedVelocity、Rect sw
CollisionItem、CollisionBatch	项: EntityId entity、Vec2 allowedPosition、normal、bool grounded、hit
TriggerHit、TriggerBatch	命中: EntityId entity、std::string triggerId、TriggerKind kind; 批次:
ChangeKind、ChangeValue	ChangeKind: Position、Velocity、Facing、Health、Energy、Active、Gr = std::variant<Vec2, int, bool, std::string, std::uint64_t, AttackInstar
StateChange、ChangeBatch	项: EntityId target、ChangeKind kind、ChangeValue value、FrameId sourc
EventKind、DomainEvent、EventBatch	EventKind: AttackStarted、Hit、Hurt、Died、PickedUp、CheckpointActi std::vector<DomainEvent> items
DamageCandidate	EntityId source、target; std::uint64_t attackId; int amount; Vec2 kn
CombatInput	worldSnapshot before; CollisionBatch resolved; IntentBatch intents; L
CombatResult	ChangeBatch changes; std::vector<DamageCandidate> damage; EventBatch e
AIResult	IntentBatch intents; ChangeBatch changes
CommitResult	worldSnapshot world; EventBatch events
RenderEntity、RenderSnapshot	可绘制实体: EntityId id、Vec2 position、ResourceId spriteKey、int anim

值匹配规则： ChangeKind::Position、Velocity 使用 Vec2；Facing、Health、Energy、JumpCount 使用 int；Active、Grounded、LevelComplete 使用 bool；EquippedAttack、Checkpoint、Inventory、Switch、Objective 使用 std::string；InvulnerableUntil、AttackReadyFrame、Target、PatrolPointIndex 使用 std::uint64_t；AttackInstance 使用同名结构体；AISate 使用同名枚举。拾取物提交后通过 Active=false 停用。任何不匹配由 applyChanges 整批拒绝。

2.3 先确定数据结构放在哪个文件

接口不是把所有内容写进一个头文件。跨 DLL 传递的值类型放在 src/contracts/KnightContracts.h；公开函数和不透明句柄的前向声明放在各模块的公开 .h；只有本模块实现能读写的结构体放在该模块的私有 .h。私有头文件不安装、不加入供其他模块使用的公开包含目录，也不能出现在其他底层模块的 #include 中。下表是目标文件布局，不描述当前仓库完成程度。

数据或对象	定义文件
Vec2、Rect、LevelDefinition、TileLayer、EntitySnapshot、WorldSnapshot、SaveData 等共享值类型	src/contracts/KnightContracts.h
GameHandle、InputState、SessionState	src/game/GamePrivate.h; KnightGame.h struct GameHandle;
EntityCore、CharacterRuntime、PlayerRuntime、EnemyRuntime、EntityRecord、World、WorldDraft	src/state/WorldPrivate.h; KnightGame.h 写 struct world;
TileRef、MapHandle	src/map/MapPrivate.h; KnightMap.h struct MapHandle;
LevelDraft	src/content/ContentPrivate.h
SaveHandle	src/save/SavePrivate.h; KnightSave.h struct SaveHandle;
GameWindow、SceneRenderer 等界面对象	src/ui/GameWindow.h、src/ui/SceneRenderer.h

KnightMotion.dll、KnightCombat.dll、KnightAI.dll 不拥有跨帧可变对象：运动结果、战斗结果和敌人意图使用 KnightContracts.h 中的值类型，本次调用结束后模块不保留输入指针。这样敌人状态、攻击实例和玩家状态始终由 world 统一管理。

2.4 地图对象：静态定义与运行索引分开

项目中“地图”有两种含义。LevelDefinition 是关卡文件解析后的静态数据，由内容模块返回；MapHandle 是地图模块从静态数据建立的运行时碰撞与触发区索引。界面的场景对象只负责画出来，不是这两者之一。

在 src/contracts/KnightContracts.h 中定义公开的静态数据，下面展示地图相关结构体的成员；Vec2、Rect、枚举和原型类型按第 2.2 节先定义：

```
namespace knight {
struct PlayerRules {
    float moveSpeed = 0, jumpSpeed = 0, gravity = 0, maxFallSpeed = 0;
    int maxJumps = 0, maxHealth = 0, maxEnergy = 0;
    Rect collider{};
    ResourceId spriteKey, defaultAttackId;
};

struct EnemyRules {
    float moveSpeed = 0, detectRadius = 0, attackRange = 0;
    FrameId attackCooldown = 0;
    int maxHealth = 0, maxEnergy = 0;
    Rect collider{};
    ResourceId spriteKey, attackId, behaviorId;
};

struct AttackRules {
    Rect hitBox{};
    int damage = 0;
    FrameId windupFrames = 0, activeFrames = 0;
    FrameId recoveryFrames = 0, cooldownFrames = 0;
};

struct TrapRules {
    int damage = 0;
    FrameId periodFrames = 0, activeFrames = 0;
};

struct PrototypeCatalog {
    PlayerRules player;
    std::unordered_map<ResourceId, EnemyRules> enemies;
    std::unordered_map<ResourceId, AttackRules> attacks;
    std::unordered_map<ResourceId, TrapRules> traps;
};
}
```

```

struct TileLayer {
    int width = 0;
    int height = 0;
    std::vector<int> tiles; // 按行存储: tiles[row * width + column]
};

struct EntitySpawn {
    EntityId id = 0;
    EntityKind kind = EntityKind::Enemy;
    ResourceId prototypeId;
    ResourceId spriteKey;
    Vec2 position{};
    Rect collider{}; // 拾取物必须填写; 敌人留空则使用原型碰撞盒
    std::vector<Vec2> patrolPoints; // 仅敌人可使用, 坐标为世界坐标
};

struct TriggerDefinition {
    std::string id; // 关卡内唯一的触发区编号
    TriggerKind kind = TriggerKind::Hazard;
    Rect area{}; // 世界坐标中的区域
    ResourceId prototypeId; // 陷阱、物品或机关的原型编号
    EntityId linkedEntity = 0; // 拾取区对应的物品实体; 其他触发区为 0
};

struct LevelDefinition {
    std::uint32_t formatVersion = 1;
    LevelId id;
    LevelId nextLevelId; // 最后一关可为空
    float tileSize = 0.0f;
    TileLayer collisionLayer;
    std::vector<TileLayer> visualLayers;
    Vec2 playerSpawn{};
    std::vector<EntitySpawn> spawns;
    std::vector<TriggerDefinition> triggers;
    std::vector<std::string> objectives;
    PrototypeCatalog prototypes;
};
}

```

碰撞层中 0 表示空格, 1 表示实心图块, 2 表示仅从上方落下时阻挡的单向平台; 其他值由内容模块拒绝。width × height 必须等于 tiles.size(), 所有图层尺寸一致, tileSize > 0。visualLayers 供业务层制作可见区域和界面绘制; collisionLayer 与 triggers 供地图模块查询。spawns 不重复放玩家: 玩家只有 playerSpawn 一个出生点, 玩家实体编号固定为 1; 敌人和拾取物由 spawns 给出不小于 2 的唯一编号。玩家精灵取自 PlayerRules.spriteKey; 敌人优先使用非空的 EntitySpawn.spriteKey, 否则使用其 EnemyRules.spriteKey; 拾取物必须填写自己的 EntitySpawn.spriteKey 和有效 collider。敌人的 EntitySpawn.collider 若宽高均为零则使用 EnemyRules.collider, 否则两者都必须大于零并作为关卡实例的覆盖值。敌人出生项的 prototypeId 必须能在 prototypes.enemies 中找到, 玩家和敌人的攻击编号必须能在 prototypes.attacks 中找到。

TriggerDefinition.prototypeId 对危险区引用 prototypes.traps 中的陷阱原型, 对拾取区表示奖励物品编号, 对门表示所需关键物品编号; 检查点和出口可为空, 其身份由触发区 id 和 kind 决定。拾取物的触发区 linkedEntity 必须指向同关的一个 Pickup 实体; 拾取后该实体被设为不活动, 业务层根据快照过滤对应触发区。非拾取触发区的 linkedEntity 必须为 0。关卡文件的路径和格式由内容模块处理, 其他模块只接收 LevelDefinition 值对象。

在 src/map/MapPrivate.h 中定义运行时结构体; 该文件包含 KnightContracts.h、<cstdint> 和 <vector>, 只允许地图 DLL 的实现包含:

```

namespace knight {
struct TileRef {
    int row = 0;
    int column = 0;
    Rect worldBounds{}; // 某个候选图块在世界中的矩形
};

struct MapHandle {
    LevelId levelId;
    float tileSize = 0.0f;
    TileLayer collisionLayer;
    std::vector<TriggerDefinition> triggers;
    // 下标仍是 row * width + column; 元素是 triggers 中的索引。
    std::vector<std::vector<std::size_t>> triggerIndicesByCell;
};
}

```

createMap(level) 复制必需的碰撞层和触发区, 建立 triggerIndicesByCell; resolveMovement(map, before, proposed) 用 collisionLayer 与 TileRef 裁决位移; queryTriggers(map, before, resolved) 先查附近格子的触发区索引, 再返回 TriggerBatch。MapHandle 不保存玩家坐标, 也不能修改 world。切关成功或加载失败时, 由业务层调用 destroyMap 释放此句柄。

2.5 游戏角色: 权威实体与只读快照分开

角色不是 Qt 的图形对象。玩家和敌人首先是**状态模块中的实体记录**; 界面绘制的角色只是这些记录在某一帧的显示结果。玩家与敌人共享位置、速度、生命、碰撞盒等字段, 同时各有自己的运行字段。

src/state/worldPrivate.h 定义权威数据, 可采用下面的组合结构; 该头文件包含 KnightContracts.h、<cstdint>、<optional>、<unordered_map> 和 <vector>:

```

namespace knight {
struct EntityCore {
    EntityId id = 0;
    EntityKind kind = EntityKind::Player;
    ResourceId prototypeId;           // 找到关卡原型参数
    Vec2 position{};
    Rect collider{};                 // 相对角色位置的碰撞盒
    ResourceId spriteKey;             // 只提供资源键，不保存图像对象
    bool active = true;
};

struct CharacterRuntime {
    Vec2 velocity{};
    int facing = 1;                  // -1 朝左, 1 朝右
    int health = 0;
    int maxHealth = 0;
    int energy = 0;
    int maxEnergy = 0;
    bool grounded = false;
    FrameId invulnerableUntil = 0;
};

struct PlayerRuntime {
    int remainingJumps = 0;
    FrameId attackReadyFrame = 0;
    ResourceId equippedAttackId;
};

struct EnemyRuntime {
    AIState state = AIState::Patrol;
    EntityId target = 0;             // 0 表示当前没有目标
    std::uint64_t patrolPointIndex = 0;
    FrameId attackReadyFrame = 0;
};

struct EntityRecord {
    EntityCore core;
    std::optional<CharacterRuntime> character;
    std::optional<PlayerRuntime> player;
    std::optional<EnemyRuntime> enemy;
};

struct World {
    LevelId levelId;
    FrameId frame = 0;
    std::unordered_map<EntityId, EntityRecord> entities;
    std::vector<AttackInstance> attacks;
    ProgressState progress;
    bool levelComplete = false;
};

struct WorldDraft {
    World candidate;                 // 验证全部变更成功后才替换权威 world
};
}

```

实体约束： `EntityKind::Player` 必须有 `character` 与 `player`、没有 `enemy`；`EntityKind::Enemy` 必须有 `character` 与 `enemy`、没有 `player`；`EntityKind::Pickup` 三项都没有。实体编号在同一世界内唯一。玩家的`最大生命`、`最大能量`、`碰撞盒`和`默认攻击`来自 `PrototypeCatalog.player`；敌人的`对应值`来自 `PrototypeCatalog.enemies[prototypeId]`，`巡逻路线`来自 `EntitySpawn.patrolPoints`。`CharacterRuntime` 保存原型确定的`上限`和`随后变化的当前值`；`位置`、`冷却`及`决策状态`都属于`运行状态`。拾取后把物品的 `core.active` 设为 `false`。已解锁能力、关键物品和永久机关属于 `world.progress`，不重复保存在角色字段里。攻击实例统一放在 `world.attacks`，避免一次挥击被错误地当成角色的永久属性。

`initializeWorld(level, restore)` 根据 `LevelDefinition.playerspawn`、`spawns` 和可选存档，创建 `world` 内的`玩家`、`敌人`和`拾取物`。`applyChanges` 不直接改原 `world`：先复制为 `worldDraft`，验证并应用整批变更，成功后一次替换；失败保留旧世界。`resetAtCheckpoint` 同样先构造候选世界，再整体替换。

状态模块通过 `readWorld` 和 `getPlayer` 把私有实体转换成 `EntitySnapshot`、`WorldSnapshot`。`EntitySnapshot` 在 `src/contracts/KnightContracts.h` 定义，是**只读传输结构体**：`id`、`kind`、`position`、`collider`、`active`、`prototypeId`、`spriteKey` 来自 `EntityCore`；`速度`和`生命`等字段来自 `CharacterRuntime`；玩家的 `remainingJumps`、`attackReadyFrame`、`equippedAttackId` 来自 `PlayerRuntime`；敌人的 `aiState`、`target`、`patrolPointIndex`、`attackReadyFrame` 来自 `EnemyRuntime`。`运动`、`地图`、`战斗`和`敌人决策`只能接收这些快照和关卡值对象，不能包含 `worldPrivate.h` 或获取 `EntityRecord*`。

下面是快照和持久化进度的目标字段，定义在 `src/contracts/KnightContracts.h`。需要先定义 `AttackInstance`、`Vec2`、`Rect` 和`各别名`、`枚举`。这些结构体是一次调用的**值副本**，调用者可以读取，但修改副本不会改变 `world`；真正更新世界只能提交 `ChangeBatch`。

```

namespace knight {
struct EntitySnapshot {
    EntityId id = 0, target = 0;
    EntityKind kind = EntityKind::Player;
    Vec2 position{}, velocity{};
    Rect collider{};
    int health = 0, energy = 0, remainingJumps = 0, facing = 1;
    bool active = true, grounded = false;
    FrameId invulnerableUntil = 0, attackReadyFrame = 0;
};
}

```



```
std::uint64_t patrolPointIndex = 0;
AIState aiState = AIState::None;
ResourceId prototypeId, spriteKey, equippedAttackId;
};

struct ProgressState {
    LevelId levelId;
    std::string checkpointId;
    int health = 0, energy = 0;
    std::vector<std::string> abilities, keyItems;
    std::vector<std::string> permanentSwitches, clearedObjectives;
};

struct WorldSnapshot {
    LevelId levelId;
    FrameId frame = 0;
    std::vector<EntitySnapshot> entities;
    std::vector<AttackInstance> attacks;
    ProgressState progress;
    bool levelComplete = false;
};
}
```

world.progress 只保存跨关或需要恢复的进度，EntityRecord.character 才是当前逻辑帧的权威生命和能量。检查点、通关或主动保存时，业务层从已提交的玩家快照同步生命和能量到 saveData.progress；加载存档时，状态模块把这两个值恢复到玩家角色。EntitySnapshot 中对非角色无意义的数字字段填零，aiState 填 None，无关的资源编号留空；读取前先检查 kind 和 active。敌人决策模块从快照读取上帧 target 与 patrolPointIndex，经 AIResult.changes 提交新值；战斗模块从 prototypeId 和 equippedAttackId 选择攻击规则。WorldSnapshot.entities 由 readWorld 生成，不能把快照当作可修改的角色对象。

2.6 其他模块自己的数据对象

模块	私有结构体及文件	必须保存的成员	创建、使用和销毁方式
游戏协调	GamePrivate.h 中的 InputState、GameHandle	InputState: 左右按住、跳跃/攻击/交互新按下标记、下一命令号；GameHandle: 会话状态、命令队列、时间累积、当前 World*/MapHandle*/SaveHandle*、关卡定义、当前槽位、画面快照、事件队列	createGame 创建；输入和每帧运行时修改；destroyGame 调用各模块销毁函数后释放
关卡内容	ContentPrivate.h 中的 LevelDraft	未完成验证的 LevelDefinition partial、std::vector<ResourceId> unresolvedRefs、文件路径及解析错误位置	loadLevel 内创建；原型解析和校验成功后只返回 LevelDefinition，草稿立即释放
存档数据库	SavePrivate.h 中的 SaveHandle	DatabasePath path、SqliteDatabase connection、int schemaVersion、连接名	openSaveStore 创建并建库/迁移；所有 SQL 只在本模块执行；closeSaveStore 关闭并释放
运动与物理	不需要跨帧私有对象；运算中可在 knightMotion.cpp 使用局部计算变量	本帧速度、加速度、建议位置	每次 proposeMotion、finishMotion 临时创建，结果写入共享 MotionBatch 或 ChangeBatch
战斗与陷阱	不需要跨帧私有对象；运算中可在 knightCombat.cpp 使用局部计算变量	本帧命中框、临时命中集合、待裁决伤害	每次公开调用临时创建，结果写入共享 CombatResult；跨帧攻击实例由 world 保存
敌人决策	不需要跨帧私有对象；运算中可在 knightAI.cpp 使用局部计算变量	本帧目标距离、决策结果	每次 decideEnemies 临时创建，结果写入共享 AIResult；跨帧决策状态由 world 保存

以下是这三类私有对象的目标字段。GamePrivate.h 仅由游戏协调 DLL 的实现包含；它可以包含各底层公开头文件来声明句柄，但不能包含底层私有头文件。SessionState 对应业务文档中的“主菜单、加载中、游玩中、已暂停、死亡中、关卡完成、退出”。

```
// src/game/GamePrivate.h
namespace knight {
enum class SessionState {
    MainMenu, Loading, Playing, Paused, Dead, LevelComplete, Exiting
};

struct InputState {
    bool leftHeld = false, rightHeld = false;
    bool jumpHeld = false, attackHeld = false, interactHeld = false;
    bool jumpPressedThisFrame = false, attackPressedThisFrame = false;
    bool interactPressedThisFrame = false;
    CommandId nextCommandId = 1;
};

struct GameHandle {
```

```
GameConfig config;
SessionState session = SessionState::MainMenu;
InputState input;
std::deque<PlayerCommand> pendingCommands;
std::uint64_t accumulatedMicroseconds = 0;
std::optional<LevelDefinition> currentLevel;
World* world = nullptr;
MapHandle* map = nullptr;
SaveHandle* save = nullptr;
SlotId currentSlot = 0;
RenderSnapshot rendered;
EventBatch queuedEvents;
};
}
```

此头文件还需包含 `<cstdint>`、`<deque>`、`<optional>`。`leftHeld/rightHeld` 在按下与释放时改变，持续移动每帧读取；`jumpHeld/attackHeld/interactHeld` 用来识别新的物理按下，三个 `PressedThisFrame` 标记仅消费一次，避免按住移动时攻击事件被覆盖。`pendingCommands` 保存已编号、标明目标帧的命令；`currentLevel` 在主菜单时为空，进入关卡后才有值。`world`、`map`、`save` 只指向其他 DLL 创建的句柄，由业务层调用对应销毁函数，绝不直接 `delete`。

```
// src/content/ContentPrivate.h
namespace knight {
struct LevelDraft {
    LevelDefinition partial;
    std::vector<ResourceId> unresolvedRefs;
    std::string sourcePathUtf8;
    int errorLine = 0, errorColumn = 0;
};
}

// src/save/SavePrivate.h: 只在 KnightSave.dll 内包含 QSql/QSqlDatabase。
namespace knight {
struct SaveHandle {
    DatabasePath path;
    std::string connectionName;
    QSqlDatabase connection;
    int schemaVersion = 0;
};
}
```

`LevelDraft` 只在 `loadLevel` 的解析、原型解析和验证过程中存在；`partial` 缺字段时不允许返回给业务层。`SaveHandle.connection` 的创建、事务、关闭和连接名释放都在存档 DLL 内完成，其他模块既不包含 `QSqlDatabase`，也不执行 SQL。

业务层持有 `World*`、`MapHandle*`、`SaveHandle*`，但这些都是**不透明句柄**：业务层可把句柄原样传回创建它的 DLL，不能读取其成员。`GameHandle` 的私有定义只供业务 DLL 使用，界面也遵守同样规则。

2.7 从文件到运行的一条具体数据路径

步骤	产生或持有的数据	结构体怎样使用
读取关卡	内容模块内部 <code>LevelDraft</code> → 公开 <code>LevelDefinition</code>	校验图块、出生点、原型和触发区；半成品不交给其他模块
建立场景	<code>LevelDefinition</code> → 地图模块的 <code>MapHandle</code> ；同一份定义 → 状态模块的 <code>World</code>	地图建立空间索引；状态建立玩家和敌人实体；两个模块不互相调用
推进一帧	<code>World</code> → <code>WorldSnapshot</code> → 运动、地图、战斗和敌人决策结果	私有角色数据只由状态模块转换为快照；各算法模块返回候选
提交结果	业务层合并 <code>ChangeBatch</code> → 状态模块的 <code>WorldDraft</code> → <code>World</code>	全部候选一次提交；失败不改变权威世界
绘制和存档	已提交 <code>WorldSnapshot</code> → <code>RenderSnapshot</code> 或 <code>SaveData</code>	界面只绘图；数据库只保存可恢复进度

2.8 所有权、校验与编译边界

创建函数返回的句柄由**同一 DLL** 的销毁函数释放；业务层不能 `delete` 句柄。私有头文件只作为所属 DLL 目标的 `PRIVATE` 源文件使用；给其他模块的编译路径只提供公开 `.h` 和 `KnightContracts.h`。`const&` 输入在调用返回后不再被引用；需保留的数据由被调用模块复制。`Result<T>::value` 只在 `status.code == Ok` 时有效。

`ChangeBatch.sourceFrame` 必须等于当前 `WorldSnapshot.frame + 1`；一次逻辑帧由业务层合并候选，只调用一次 `applyChanges`，成功后世界帧号增加 1。存档只读取提交后的世界。数据库写入失败时旧存档不变。坐标必须有限，碰撞盒宽高必须大于零。若改变上述结构体的公开字段或类型，所有使用它们的目标必须用同一套工具链重新构建。

3 界面调用业务层：KnightGame.h

以下公开函数**仅供界面调用**。它们声明在 `src/game/KnightGame.h`，使用 `KNIGHTGAME_API` 导出；七个底层模块不能反向调用。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<GameHandle*> createGame(const GameConfig& config) noexcept</code>	<code>config: GameConfig</code> ，含内容目录、数据库路径、首关编号和契约版本	<code>Result<GameHandle*></code> ；成功时 <code>value</code> 是业务句柄，失败时看 <code>status</code>	程序启动，建立会话并打开存档
<code>void destroyGame(GameHandle* game) noexcept</code>	<code>game: GameHandle*</code> ，可为空，只能是本 DLL 创建的句柄	<code>void</code> ；释放会话、世界、地图和存档连接，原指针失效	从菜单退出程序
<code>Status submitCommand(GameHandle* game, const PlayerCommand& command) noexcept</code>	<code>game: GameHandle*</code> 当前会话； <code>command: PlayerCommand</code> 动作、阶段、方向和可选槽位	<code>Status</code> ；成功表示已验证、编号并入队，不表示动作已生效	键盘、鼠标、菜单每产生一次动作时
<code>Status advanceElapsed(GameHandle* game, std::uint64_t elapsedMicroseconds) noexcept</code>	<code>game: GameHandle*</code> ； <code>elapsedMicroseconds: uint64_t</code> 自上次计时唤醒的微妙数	<code>Status</code> ；内部推进零到五个固定逻辑帧，完成后状态保存在会话内	界面定时器唤醒时
<code>Result<RenderSnapshot> readRenderSnapshot(GameHandle* game) noexcept</code>	<code>game: GameHandle*</code>	<code>Result<RenderSnapshot></code> ；成功时 <code>value</code> 是完整画面副本	界面绘制菜单或游戏画面时
<code>Result<EventBatch> takeEvents(GameHandle* game) noexcept</code>	<code>game: GameHandle*</code>	<code>Result<EventBatch></code> ；成功时 <code>value</code> 是尚未消费的事件，取出后队列清空；失败不清空	播放命中、死亡、保存结果等一次性反馈

以下函数定义在 `src/game/knightGame.cpp`，**只由业务 DLL 内部调用**。`GameHandle&` 是输入兼输出：函数可改变会话私有状态，但不把它传给底层模块。

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>Status normalizeCommand(GameHandle& game, const PlayerCommand& raw, PlayerCommand& normalized)</code>	<code>game</code> ：当前输入状态； <code>raw: PlayerCommand</code> ：界面命令； <code>normalized: PlayerCommand&</code> ：调用方提供的输出位置	返回 <code>Status</code> ；成功写入 <code>normalized</code> ，更新按住状态并分配命令号和目标帧	<code>submitCommand</code> 收到任意输入时
<code>Status handleMenuCommand(GameHandle& game, const PlayerCommand& command)</code>	<code>game</code> ：会话； <code>command</code> ：已编号的菜单命令	返回 <code>Status</code> ；成功时改变会话状态，必要时调用内容、状态、地图或存档公开接口	<code>advanceElapsed</code> 在玩法帧之前处理开始、继续、暂停、重试等
<code>Status runFixedFrame(GameHandle& game)</code>	<code>game</code> ：持有当前世界、地图、输入和关卡定义	返回 <code>Status</code> ；成功时推进一帧、提交世界并更新画面和事件	<code>advanceElapsed</code> 每累计 1/60 秒时
<code>Result<SaveData> makeSaveData(const WorldSnapshot& committed, SlotId slotId)</code>	<code>committed: WorldSnapshot</code> 已提交世界； <code>slotId: SlotId</code> 当前槽位	<code>Result<SaveData></code> ；成功时 <code>value</code> 是可写入数据库的进度副本	检查点、通关、保存并返回菜单时
<code>Result<RenderSnapshot> makeRenderSnapshot(const WorldSnapshot& committed, const LevelDefinition& level)</code>	<code>committed</code> 已提交世界； <code>level</code> 已验证关卡定义	<code>Result<RenderSnapshot></code> ；成功时 <code>value</code> 是界面绘制数据	初始化关卡、每帧提交、复活后
<code>Status publishEvents(GameHandle& game, const EventBatch& confirmedEvents)</code>	<code>game</code> 事件队列； <code>confirmedEvents</code> 已确认的玩法事件或存档结果事件	返回 <code>Status</code> ；成功将事件编号后入队，供 <code>takeEvents</code> 取走	世界提交成功后发布玩法事件；数据库返回后发布保存结果事件

4 业务层调用底层模块

每个小节第一张表是**公开函数**：只允许 `KnightGame.dll` 调用，声明在对应模块的公开 `.h`。第二张表是**内部函数**：只允许同一个 DLL 的 `.cpp` 调用，签名中的私有类型见 2.3。函数栏不记录开发状态。

4.1 对象与状态：KnightState.h

公开声明位于 `src/state/knightState.h`，使用 `KNIGHTSTATE_API` 导出。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>world* createWorld() noexcept</code>	无参数	<code>world*</code> ；成功为候选世界句柄，内存不足返回 <code>nullptr</code>	新游戏、读档、切关前创建候选世界
<code>void destroyWorld(world* world) noexcept</code>	<code>world: world*</code> 本模块创建的句柄，可为空	<code>void</code> ；释放后原指针失效	退出、切关成功后释放旧世界、失败时回滚
<code>bool getPlayer(world* world, EntitySnapshot* output) noexcept</code>	<code>world: world*</code> 当前世界； <code>output: EntitySnapshot*</code> 调用方提供的可写地址，不能为空	<code>bool</code> ；返回 <code>true</code> 时把玩家快照写入 <code>*output</code> ；返回 <code>false</code> 时 <code>*output</code> 不可使用	业务层只需要玩家单体数据时；完整逻辑帧仍使用 <code>readWorld</code>
<code>Status initializeWorld(world* world, const LevelDefinition& level, const SaveData* restore) noexcept</code>	<code>world: world*</code> 候选世界； <code>level: LevelDefinition</code> 已验证关卡； <code>restore: SaveData*</code> 可为空，空表示新游戏	<code>Status</code> ；成功建立实体与进度，失败保持旧候选内容	新游戏、读档、切关时
<code>Result<worldSnapshot> readWorld(world* world) noexcept</code>	<code>world: world*</code> 当前世界	<code>Result<worldSnapshot></code> ； <code>value</code> 是独立的完整世界副本	每帧开始、存档前、复活后
<code>Result<CommitResult> applyChanges(world* world, const ChangeBatch& changes) noexcept</code>	<code>world: world*</code> 权威世界； <code>changes: ChangeBatch</code> 本帧合并后的全部变更	<code>Result<CommitResult></code> ； <code>value.world</code> 为新快照， <code>value.events</code> 为提交事件；失败整批不生效	每逻辑帧唯一一次状态提交
<code>Status resetAtCheckpoint(world* world, const LevelDefinition& level, const SaveData& progress) noexcept</code>	<code>world</code> 当前世界； <code>level</code> 本关规则； <code>progress: SaveData</code> 检查点和永久进度	<code>Status</code> ；成功重建世界，失败保持原世界	死亡复活、暂停菜单重试

内部函数定义在 `src/state/KnightState.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>Result<WorldDraft> buildWorldDraft(const LevelDefinition& level, const SaveData* restore)</code>	<code>level</code> 关卡出生点和实体定义； <code>restore</code> 可空进度	<code>Result<worldDraft></code> ，私有候选实体注册表	<code>initializeWorld</code> 、 <code>resetAtCheckpoint</code> 替换权威世界前
<code>Status validateChangeBatch(const world& world, const ChangeBatch& changes)</code>	<code>world</code> 当前实体注册表； <code>changes</code> 候选变更	<code>Status</code> ，首个非法实体、字段类型或来源帧的错误	<code>applyChanges</code> 修改任何状态前
<code>Status applyChangeToDraft(WorldDraft& draft, const stateChange& change)</code>	<code>draft</code> 私有候选世界； <code>change</code> 单条已校验变更	<code>Status</code> ；成功修改草稿，不修改权威 <code>world</code>	<code>applyChanges</code> 遍历本帧变更时
<code>Result<worldSnapshot> buildSnapshot(const world& world)</code>	<code>world</code> 已确认世界	<code>Result<worldSnapshot></code> 独立副本	<code>readWorld</code> 和提交完成后
<code>Result<EventBatch> buildCommitEvents(const worldSnapshot& before, const worldSnapshot& after)</code>	<code>before</code> 旧快照； <code>after</code> 新快照	<code>Result<EventBatch></code> ，死亡、拾取等提交事件	<code>applyChanges</code> 提交草稿前构造结果时

4.2 地图与碰撞：KnightMap.h

公开声明位于 `src/map/KnightMap.h`，使用 `KNIGHTMAP_API` 导出。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<MapHandle*> createMap(const LevelDefinition& level) noexcept</code>	level 已验证的碰撞层、图块大小和触发区	Result<MapHandle*>; value 是地图索引句柄	新游戏、读档、切关时建图
<code>void destroyMap(MapHandle* map) noexcept</code>	map: MapHandle* 本模块创建的句柄，可为空	void; 释放索引后指针失效	退出、切关或加载失败回滚
<code>Result<CollisionBatch> resolveMovement(MapHandle* map, const WorldSnapshot& before, const MotionBatch& proposed) noexcept</code>	map 本关地图; before 旧实体碰撞盒; proposed 运动模块经业务层传来的建议位移	Result<CollisionBatch>; 每实体允许位置、法线、落地与撞顶标志	每帧运动提案完成后
<code>Result<TriggerBatch> queryTriggers(MapHandle* map, const WorldSnapshot& before, const CollisionBatch& resolved) noexcept</code>	map 触发区索引; before 实体身份和规则; resolved 裁决后位置	Result<TriggerBatch>; 危险区、物品、检查点、门、出口命中	每帧碰撞裁决后

内部函数定义在 `src/map/KnightMap.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>Status buildSpatialIndex(MapHandle& map, const LevelDefinition& level)</code>	map 私有索引容器; level 图块层与触发区	Status; 成功填充 map 的图块和触发区索引	createMap 建立句柄后
<code>std::vector<TileRef> collectNearbyTiles(const MapHandle& map, const Rect& sweptBounds)</code>	map 索引; sweptBounds 实体本帧扫过的世界区域	实心图块引用列表, TileRef 为地图 DLL 私有类型	resolveMovement 缩小碰撞候选集
<code>CollisionItem sweepAxis(const EntitySnapshot& entity, const MotionItem& motion, const std::vector<TileRef>& tiles, bool horizontal)</code>	entity 旧碰撞盒; motion 建议运动; tiles 邻近图块; horizontal 是否横向裁决	CollisionItem, 该轴允许位置和碰撞法线	resolveMovement 先横向、后纵向裁决
<code>TriggerBatch overlapTriggers(const MapHandle& map, const CollisionBatch& resolved)</code>	map 触发区索引; resolved 裁决后实体位置	TriggerBatch 命中列表	queryTriggers 逐实体判定触发区

4.3 运动与物理：KnightMotion.h

公开声明位于 `src/motion/KnightMotion.h`，使用 `KNIGHTMOTION_API` 导出。运动规则来自 `LevelDefinition.prototype`，所以公开函数必须把关卡定义作为参数。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<MotionBatch> proposeMotion(const WorldSnapshot& before, const LevelDefinition& level, const IntentBatch& intents, float fixedSeconds) noexcept</code>	before 旧位置和速度; level 玩家/敌人运动参数; intents 本帧意图; fixedSeconds: float 固定步长，首版为 1/60	Result<MotionBatch>; 建议位置、速度和扫过区域，不修改世界	输入和敌人决策之后
<code>Result<ChangeBatch> finishMotion(const MotionBatch& proposed, const CollisionBatch& resolved) noexcept</code>	proposed 运动提案; resolved 地图裁决结果	Result<ChangeBatch>; 最终位置、速度、落地及跳跃次数候选	地图裁决后、战斗前

内部函数定义在 `src/motion/KnightMotion.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>float chooseAcceleration(const EntitySnapshot& entity, const Intent& intent, const LevelDefinition& level)</code>	<code>entity</code> 当前速度和类别； <code>intent</code> 方向； <code>level</code> 对应原型	<code>float</code> 水平加速度	<code>proposeMotion</code> 计算左右移动时
<code>Vec2 applyGravityAndJump(const EntitySnapshot& entity, const Intent& intent, const LevelDefinition& level, float fixedSeconds)</code>	<code>entity</code> 落地、跳跃次数与旧速度； <code>intent</code> 跳跃请求； <code>level</code> 重力与跳速； <code>fixedSeconds</code> 步长	<code>Vec2</code> 建议速度	<code>proposeMotion</code> 计算起跳和下落时
<code>Vec2 clampVelocity(const Vec2& velocity, EntityKind kind, const LevelDefinition& level)</code>	<code>velocity</code> 原建议速度； <code>kind</code> 实体类别； <code>level</code> 速度上限	<code>Vec2</code> 截断后的合法速度	<code>proposeMotion</code> 输出提案前
<code>std::vector<StateChange> reconcileCollision(const MotionItem& proposed, const CollisionItem& resolved)</code>	<code>proposed</code> 单实体建议位置/速度； <code>resolved</code> 单实体碰撞裁决	状态变更列表：位置、速度、落地等	<code>finishMotion</code> 逐实体生成候选

4.4 战斗与陷阱：KnightCombat.h

公开声明位于 `src/combat/KnightCombat.h`，使用 `KNIGHTCOMBAT_API` 导出。战斗只计算候选，不直接调用状态 DLL 扣血。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<CombatResult> evaluateAttacks(const CombatInput& input) noexcept</code>	<code>input</code> ：CombatInput，含旧世界、地图裁决后位置、攻击意图和关卡攻击原型	<code>Result<CombatResult></code> ； <code>value.changes</code> 为攻击阶段/冷却候选， <code>value.damage</code> 为命中候选， <code>value.events</code> 为攻击事件候选	玩家或敌人新攻击，以及已有攻击实例推进的每一帧
<code>Result<CombatResult> evaluateTraps(const CombatInput& input, const TriggerBatch& triggers) noexcept</code>	<code>input</code> 旧世界和陷阱原型； <code>triggers</code> ：TriggerBatch 地图经业务层传来的危险区命中	<code>Result<CombatResult></code> ；陷阱伤害、机关相位与事件候选	实体接触尖刺或周期机关的帧
<code>Result<CombatResult> resolveDamage(const WorldSnapshot& before, const CombatResult& candidates) noexcept</code>	<code>before</code> 受击前生命、无敌截止帧； <code>candidates</code> 业务层合并的攻击与陷阱结果	<code>Result<CombatResult></code> ；去重后的扣血、击退、无敌时间、死亡和事件候选	本帧全部伤害事实到齐之后、 <code>applyChanges</code> 之前

内部函数定义在 `src/combat/KnightCombat.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>AttackPhase advanceAttackPhase(const AttackInstance& attack, FrameId frame, const AttackRules& rules)</code>	<code>attack</code> 开始帧； <code>frame</code> 当前帧； <code>rules</code> 起手/生效/收招长度	<code>AttackPhase</code> 当前阶段	<code>evaluateAttacks</code> 推进每个攻击实例
<code>bool overlapHitBox(const Vec2& attackerPosition, const Rect& hitBox, const EntitySnapshot& target)</code>	攻击者位置、相对攻击框、目标碰撞盒	<code>bool</code> 是否空间重叠	<code>evaluateAttacks</code> 仅在生效阶段
<code>bool alreadyHitTarget(const AttackInstance& attack, EntityId target)</code>	<code>attack</code> 已命中集合； <code>target</code> 待检测目标	<code>bool</code> 是否重复命中	<code>evaluateAttacks</code> 同一挥击扣血前
<code>bool isTrapActive(const TrapRules& rules, FrameId frame)</code>	<code>rules</code> 周期与生效帧； <code>frame</code> 当前逻辑帧	<code>bool</code> 当前是否危险	<code>evaluateTraps</code> 判定周期陷阱
<code>std::vector<DamageCandidate> dedupeAndApplyImmunity(const WorldSnapshot& before, const std::vector<DamageCandidate>& candidates)</code>	<code>before</code> 目标无敌状态； <code>candidates</code> 同帧所有攻击/陷阱命中	去重且通过无敌规则的伤害列表	<code>resolveDamage</code> 生成生命变更前

4.5 敌人决策：KnightAI.h

公开声明位于 `src/ai/KnightAI.h`，使用 `KNIGHTAI_API` 导出。敌人状态保存在世界快照和状态候选中，本模块不保存上帧私有位置。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<AIResult> decideEnemies(const WorldSnapshot& before, const LevelDefinition& level, FrameId frame) noexcept</code>	<code>before</code> 玩家与敌人快照； <code>level</code> 索敌、巡逻和攻击原型； <code>frame</code> 当前逻辑帧	<code>Result<AIResult></code> ； <code>value.intents</code> 为敌人移动/攻击意图， <code>value.changes</code> 为决策状态候选	每帧运动计算之前

内部函数定义在 `src/ai/knightAI.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>EntityId selectTarget(const EntitySnapshot& enemy, const WorldSnapshot& world, const EnemyRules& rules)</code>	<code>enemy</code> 位置； <code>world</code> 玩家等实体； <code>rules</code> 索敌范围	<code>EntityId</code> 目标编号； 零表示没找到	<code>decideEnemies</code> 判断是否发现玩家
<code>AIState advanceDecisionState(const EntitySnapshot& enemy, EntityId target, const WorldSnapshot& world, const EnemyRules& rules, FrameId frame)</code>	当前敌人状态、目标、世界距离、规则、逻辑帧	<code>AIState</code> 下一决策状态	<code>decideEnemies</code> 在巡逻、追击和攻击间转换
<code>Intent chooseEnemyIntent(const EntitySnapshot& enemy, AIState state, EntityId target, const WorldSnapshot& world, const LevelDefinition& level)</code>	敌人快照、新决策状态、目标和关卡规则	<code>Intent</code> 水平方向或攻击请求	<code>decideEnemies</code> 输出敌人动作前

4.6 关卡与配置： `KnightContent.h`

公开声明位于 `src/content/KnightContent.h`，使用 `KNIGHTCONTENT_API` 导出。业务层只获取完整、已验证的关卡，不接收解析半成品。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<LevelDefinition> loadLevel(const ContentRoot& root, const LevelId& id) noexcept</code>	<code>root</code> 内容根目录； <code>id</code> 稳定关卡编号，不是任意文件路径	<code>Result<LevelDefinition></code> ；成功时 <code>value</code> 含图层、出生点、触发区和已解析原型，失败时 <code>status</code> 指明文件/对象/字段	新游戏、读档、存档、切关、重试

内部函数定义在 `src/content/knightContent.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>Result<LevelDraft> parseLevelFile(const ContentRoot& root, const LevelId& id)</code>	内容目录、稳定关卡编号	<code>Result<LevelDraft></code> 未验证的私有关卡草稿	<code>loadLevel</code> 读取文件后
<code>Result<PrototypeCatalog> loadPrototypes(const ContentRoot& root, const LevelDraft& draft)</code>	内容目录、草稿中引用的原型编号	<code>Result<PrototypeCatalog></code> 完整数值原型	<code>loadLevel</code> 解析关卡引用时
<code>Status validateLevel(const LevelDraft& draft, const PrototypeCatalog& prototypes)</code>	草稿及解析后的原型	<code>Status</code> ；检查图层尺寸、出生点、实体编号、出口、资源引用	<code>loadLevel</code> 返回关卡前
<code>Result<LevelDefinition> resolveResourceKeys(const LevelDraft& draft, const PrototypeCatalog& prototypes)</code>	已验证草稿和原型	<code>Result<LevelDefinition></code> ，可供其他模块直接使用的值对象	<code>loadLevel</code> 完成完整校验后

4.7 存档与数据库： `KnightSave.h`

公开声明位于 `src/save/knightSave.h`，使用 `KNIGHTSAVE_API` 导出。数据库连接和 SQL 只在本 DLL 内；不向业务层暴露 `QSqlDatabase`。

函数声明	输入参数：类型和意义	输出：具体类型和意义	使用场景
<code>Result<SaveHandle*> openSaveStore(const DatabasePath& path) noexcept</code>	<code>path</code> 当前用户数据库文件路径	<code>Result<SaveHandle*></code> ； <code>value</code> 是私有连接句柄；失败提供错误码	程序启动时建库或打开数据库
<code>void closeSaveStore(SaveHandle* store) noexcept</code>	<code>store</code> : <code>SaveHandle*</code> 本模块句柄，可为空	<code>void</code> ；关闭连接，原指针失效	程序退出、创建会话失败回滚
<code>Result<SlotId> createSlot(SaveHandle* store, const SlotSeed& seed) noexcept</code>	<code>store</code> 有效连接； <code>seed</code> 首次初始进度	<code>Result<SlotId></code> ； <code>value</code> 为新槽位号，失败事务回滚	点击开始、候选世界和地图均建立后
<code>Result<SaveData> readSlot(SaveHandle* store, SlotId id) noexcept</code>	<code>store</code> 有效连接； <code>id</code> 用户选择的槽位号	<code>Result<SaveData></code> ； <code>value</code> 是完整持久化副本	点击继续游戏
<code>Status writeSlot(SaveHandle* store, const SaveData& data) noexcept</code>	<code>store</code> 有效连接； <code>data</code> 从已提交世界生成的进度副本	<code>Status</code> ；成功整笔提交，失败回滚并保留旧存档	检查点、通关、保存并返回
<code>Result<SlotSummaryBatch> listSlots(SaveHandle* store) noexcept</code>	<code>store</code> 有效连接	<code>Result<SlotSummaryBatch></code> ； <code>value.slots</code> 是菜单摘要	主菜单列出存档

内部函数定义在 `src/save/KnightSave.cpp`：

内部函数签名	输入参数：类型和意义	输出或副作用	谁在什么场景调用
<code>Status createSchema(SaveHandle& store)</code>	<code>store</code> 私有连接	<code>Status</code> ；成功创建所需表与版本记录	<code>openSaveStore</code> 检查数据库为空时
<code>Status migrateSchema(SaveHandle& store, int currentVersion)</code>	<code>store</code> 私有连接； <code>currentVersion</code> 当前模式版本	<code>Status</code> ；支持的旧版事务迁移，不支持返回版本错误	<code>openSaveStore</code> 发现旧模式时
<code>Status validateSaveData(const SaveData& data)</code>	<code>data</code> 刚读取或待写入的进度	<code>Status</code> ；检查版本、槽位和必要字段	<code>readSlot</code> 返回前、 <code>writeSlot</code> 写入前
<code>Result<SaveData> readSlotTables(SaveHandle& store, SlotId id)</code>	<code>store</code> 私有连接； <code>id</code> 槽位号	<code>Result<SaveData></code> 多表拼成的进度副本	<code>readSlot</code> 查询各关联表时
<code>Status writeTablesInTransaction(SaveHandle& store, const SaveData& data)</code>	<code>store</code> 私有连接； <code>data</code> 完整进度	<code>Status</code> ；全部表写完才提交，否则回滚	<code>createSlot</code> 、 <code>writeSlot</code> 执行原子写入时

5 两人并行开发时如何对接

1. 先共同确定 `KnightContracts.h` 的结构体字段、枚举值、单位和契约版本，再确定第 3、4 节公开函数的签名。任何公开类型或签名变化，两人一起更新文档和头文件。
2. 一个模块的内部函数只由该模块实现文件使用；业务层只引用公开头文件。各底层 CMake 目标的链接项不得出现其他底层 DLL 的导入库。
3. 对方可以按表中的具体输入构造测试值，检查 `Status`，再检查 `Result<T>::value`。返回的快照、批次和关卡定义都是调用方持有的副本；不跨 DLL 传私有容器或私有对象指针。
4. 业务调用顺序见[业务逻辑与数据流](#)。例如：`readWorld` → `decideEnemies` → `proposeMotion` → `resolveMovement` → `queryTriggers` → `finishMotion` → `evaluateAttacks/evaluateTraps` → `resolveDamage` → `applyChanges`，每一步的返回值都先回到业务层。